

[Course Home \(/learn/data-manipulation/home/welcome\)](#) > [Week 3 \(/learn/data-manipulation/home/week/3\)](#) > [Assignment 3: MapReduce](#)

Assignment: Thinking in MapReduce

✓ Passed but not verified · 50/60 points

Deadline Pass this assignment by October 4, 11:59 PM PDT.

Instructions ([/learn/data-manipulation/programming/Dp7ql/thinking-in-mapreduce](#))

[My submission \(/learn/data-manipulation/programming/Dp7ql/thinking-in-mapreduce/submission\)](#)

[Discussions \(/learn/data-manipulation/programming/Dp7ql/thinking-in-mapreduce/discussions\)](#)

Frontmatter

In this assignment, you will be designing and implementing MapReduce algorithms for a variety of common data processing tasks.

The MapReduce programming model (and a corresponding system) was proposed in a 2004 paper from a team at Google as a simpler abstraction for processing very large datasets in parallel. The goal of this assignment is to give you experience "thinking in MapReduce." We will be using small datasets that you can inspect directly to determine the correctness of your results and to internalize how MapReduce works. In the next assignment, you will have the opportunity to use a MapReduce-based system to process the very large datasets for which it was designed.

As always, the first thing to do is to update your provided course materials using git pull. These resources may have changed since the last time you interacted with the `datasci_course_materials` repository. (Review the github instructions in the course materials if necessary).

Next, review the lectures to make sure you understand the programming model.

Python MapReduce Framework

You will be provided with a python library called `MapReduce.py` that implements the MapReduce programming model. The framework faithfully implements the MapReduce programming model, but it executes entirely on a single machine -- it does not involve parallel computation.

Here is the word count example discussed in class implemented as a MapReduce program using the framework:

```
# Part 1
mr = MapReduce.MapReduce()

# Part 2
def mapper(record):
    # key: document identifier
    # value: document contents
    key = record[0]
    value = record[1]
    words = value.split()
    for w in words:
        mr.emit_intermediate(w, 1)

# Part 3
def reducer(key, list_of_values):
    # key: word
    # value: list of occurrence counts
    total = 0
    for v in list_of_values:
        total += v
    mr.emit((key, total))

# Part 4
inputdata = open(sys.argv[1])
mr.execute(inputdata, mapper, reducer)
```

In Part 1, we create a MapReduce object that is used to pass data between the map function and the reduce function; you won't need to use this object directly.

In Part 2, the mapper function tokenizes each document and emits a key-value pair. The key is a word formatted as a string and the value is the integer 1 to indicate an occurrence of word.

In Part 3, the reducer function sums up the list of occurrence counts and emits a count for word. Since the mapper function emits the integer 1 for each word, each element in the `list_of_values` is the integer 1.

The list of occurrence counts is summed and a (word, total) tuple is emitted where word is a string and total is an integer.

In Part 4, the code loads the json file and executes the MapReduce query which prints the result to stdout.

Submission Details

For each problem, you will turn in a python script, similar to wordcount.py, that solves the problem using the supplied MapReduce framework.

When testing, make sure MapReduce.py is in the same directory as the solution script.

Solution data will be provided for each problem in the solutions folder.

Your python submission scripts are required to have a mapper function that accepts at least 1 argument and a reducer function that accepts at least 2 arguments. Your submission is also required to have a global variable named mr which points to a MapReduce object. If you solve the problems by simply replacing the mapper and reducer functions in wordcount.py, then this condition will be satisfied automatically.

Problem 1

Create an Inverted index. Given a set of documents, an inverted index is a dictionary where each word is associated with a list of the document identifiers in which that word appears.

Mapper Input

The input is a 2-element list: [document_id, text], where document_id is a string representing a document identifier and text is a string representing the text of the document. The document text may have words in upper or lower case and may contain punctuation. You should treat each token as if it was a valid word; that is, you can just use value.split() to tokenize the string.

Reducer Output

The output should be a (word, document ID list) tuple where word is a String and document ID list is a list of Strings.

You can test your solution to this problem using books.json:

```
python inverted_index.py books.json
```

You can verify your solution against inverted_index.json.

Problem 2

Implement a relational join as a MapReduce query

Consider the following query:

```
SELECT *
FROM Orders, LineItem
WHERE Order.order_id = LineItem.order_id
```

Your MapReduce query should produce the same result as this SQL query executed against an appropriate database.

You can consider the two input tables, Order and LineItem, as one big concatenated bag of records that will be processed by the map function record by record.

Map Input

Each input record is a list of strings representing a tuple in the database. Each list element corresponds to a different attribute of the table

The first item (index 0) in each record is a string that identifies the table the record originates from. This field has two possible values:

- "line_item" indicates that the record is a line item.
- "order" indicates that the record is an order.

The second element (index 1) in each record is the order_id.

LineItem records have 17 attributes including the identifier string.

Order records have 10 elements including the identifier string.

Reduce Output

The output should be a joined record: a single list of length 27 that contains the attributes from the order record followed by the fields from the line item record. Each list element should be a string.

You can test your solution to this problem using records.json:

```
$ python join.py records.json
```

You can compare your solution with join.json.

Problem 3

Consider a simple social network dataset consisting of a set of key-value pairs (person, friend) representing a friend relationship between two people. Describe a MapReduce algorithm to count the number of friends for each person.

Map Input

Each input record is a 2 element list [personA, personB] where personA is a string representing the name of a person and personB is a string representing the name of one of personA's friends. Note that it may or may not be the case that the personA is a friend of personB.

Reduce Output

The output should be a pair (person, friend_count) where person is a string and friend_count is an integer indicating the number of friends associated with person.

You can test your solution to this problem using friends.json:

```
$ python friend_count.py friends.json
```

You can verify your solution by comparing your result with the file friend_count.json.

Problem 4

The relationship "friend" is often symmetric, meaning that if I am your friend, you are my friend. Implement a MapReduce algorithm to check whether this property holds. Generate a list of all non-symmetric friend relationships.

Map Input

Each input record is a 2 element list [personA, personB] where personA is a string representing the name of a person and personB is a string representing the name of one of personA's friends. Note that it may or may not be the case that the personA is a friend of personB.

Reduce Output

The output should be all pairs (friend, person) such that (person, friend) appears in the dataset but (friend, person) does not.

You can test your solution to this problem using friends.json:

```
$ python asymmetric_friendships.py friends.json
```

You can verify your solution by comparing your result with the file asymmetric_friendships.json.

Problem 5

Consider a set of key-value pairs where each key is sequence id and each value is a string of nucleotides, e.g., GCTTCCGAAATGCTCGAA....

Write a MapReduce query to remove the last 10 characters from each string of nucleotides, then remove any duplicates generated.

Map Input

Each input record is a 2 element list [sequence id, nucleotides] where sequence id is a string representing a unique identifier for the sequence and nucleotides is a string representing a sequence of nucleotides

Reduce Output

The output from the reduce function should be the unique trimmed nucleotide strings.

You can test your solution to this problem using dna.json:

```
$ python unique_trims.py dna.json
```

You can verify your solution by comparing your result with the file unique_trims.json.

Problem 6

Assume you have two matrices A and B in a sparse matrix format, where each record is of the form i, j, value. Design a MapReduce algorithm to compute the matrix multiplication A x B

Map Input

The input to the map function will be a row of a matrix represented as a list. Each list will be of the form [matrix, i, j, value] where matrix is a string and i, j, and value are integers.

The first item, matrix, is a string that identifies which matrix the record originates from. This field has two possible values: "a" indicates that the record is from matrix A and "b" indicates that the record is from matrix B.

Reduce Output

The output from the reduce function will also be a row of the result matrix represented as a tuple. Each tuple will be of the form (i, j, value) where each element is an integer.

You can test your solution to this problem using matrix.json:



```
$ python multiply.py matrix.json
```

You can verify your solution by comparing your result with the file multiply.json.

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submission" tab.

◀ (/learn/data-manipulation/lecture/nfUMj/rdbms-vs-hadoop-

> (/learn/data-manipulation/lecture/1r2MM/nosql-context-and-

(https://accounts.coursera.org/login?return_to=https://learner.coursera.help/hc)

roadmap)